Browser History API

Problem: Build a single-page app with client-side navigation on top of the browser History API (`pushState` , `replaceState` , `popstate`). It needs a browser-like toolbar (Back, Forward, Reload, address bar), a panel that shows the history stack with the current entry highlighted, links that change the URL without a page reload, dynamic routes (`/products/:id`), a 404 page, URL-driven search (`?q=`), and a history that survives a page reload.

Tech stack: React (Vite, JavaScript, Tailwind CSS v4) — default, not given in the task

Time limit: 60 minutes — default, not given in the task

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements**: what it must do and what is out of scope. Write the assumptions down.
2. **List the interactions**: every click, key and browser event and what should happen. This is the checklist.
3. **Sketch the architecture**: components and modules, one job each.
4. **Design the state**: the smallest source of truth, and who owns it. Everything else is derived.
5. **Define contracts**: data shapes, function signatures, API mocks.
6. **Trace one flow end to end** before writing code.
7. **Find the hard parts** early and decide how to solve them.
8. **Build in layers**: smallest working version first, then add one working layer at a time.
9. **Cover states and edge cases**: empty, error, boundaries, rapid clicks, reload.
10. **Wrap up**: explain trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Use cases, architecture, state design	7 min
Pure history stack + route matcher	8 min
History API adapter + hook (push / popstate)	12 min
Link, routes, pages, 404	10 min
Toolbar, address bar, history panel	10 min
Search with replaceState, reload persistence, edge cases	5 min
Wrap-up and trade-offs	3 min

Think out loud

Say every decision before you type it: "The browser does not let me read its history list, so I will mirror it myself and tag each entry with its index." The interviewer grades the reasoning as much as the code. If you run out of time, a clear plan for the missing part still earns points.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Clicking an in-app link changes the URL and the page **without a full reload** (`history.pushState`).
- The browser's own Back / Forward buttons (and Alt+← / Alt+→) work and show the right page (`popstate`).
- In-app **Back** and **Forward** buttons, disabled when there is nowhere to go.
- **Address bar**: shows the current path, the user can type a path and press Enter to go there.
- **Reload** button: reloads the page and lands on the same page with the same history.
- **History panel**: the list of entries in this tab, current one highlighted, forward entries greyed; clicking an entry jumps there.
- Routes: `/` , `/products` , `/products/:id` , `/about` , anything else is a **404**.
- Search on `/products` stored in the URL (`?q=`) using `replaceState` , so typing does not flood the history.
- A new visit after going back **drops the forward entries** (same as a real browser).

Non-functional requirements

Area	What it means here
Performance	Navigation is instant (no network, no reload). Only one re-render per navigation.
Scalability	Route table is data; adding a page is one line. Route matching is O(routes × segments).
Error handling	Unknown path → 404 page. Unknown product id → 404. Invalid / external address → inline error. Blocked `sessionStorage` → app still works, only reload restore is lost.
Accessibility	Links are real `<a href>` (keyboard, screen reader, "open in new tab"). Buttons have `aria-label` , disabled states, `aria-current="page"` on the active nav link and history entry, `role="alert"` on the address error.
Reusability	`historyStack.js` and `matchPath.js` are pure and framework-free; `browserHistory` is a tiny store any UI can subscribe to.
Extensibility	Easy to add guards (before push), nested routes, scroll restoration, `navigate(-n)` .

Clarifying questions to ask the interviewer

1. Should we use the real History API (URL changes) or only simulate a browser in memory? → *Real API; the in-memory stack is just a mirror.*
2. Can we use a library like React Router? → *No, build the core yourself.*
3. Does the history list need to survive a page reload? → *Yes, for the same tab.*
4. Should typing in the search box create a history entry per keystroke? → *No, the URL should update but history should not flood.*
5. Do we need nested routes, route guards, or lazy loading? → *Not now, mention them.*
6. Is the server configured to serve `index.html` for every path (SPA fallback)? → *Assume yes (Vite dev does this).*

Assumptions we will build with

- Real `window.history` is the source of truth; we keep a **mirror** of the entry list because the browser does not expose it.
- Each entry is a path string (`pathname + search`). No hash routing.
- History is per tab, so the mirror goes in `sessionStorage` (same lifetime as the tab's history).
- Visiting the URL you are already on does nothing (no duplicate entry).
- Only same-origin paths can be typed in the address bar.
- Data is a static product list; no API.

2. Identify interactions / use cases

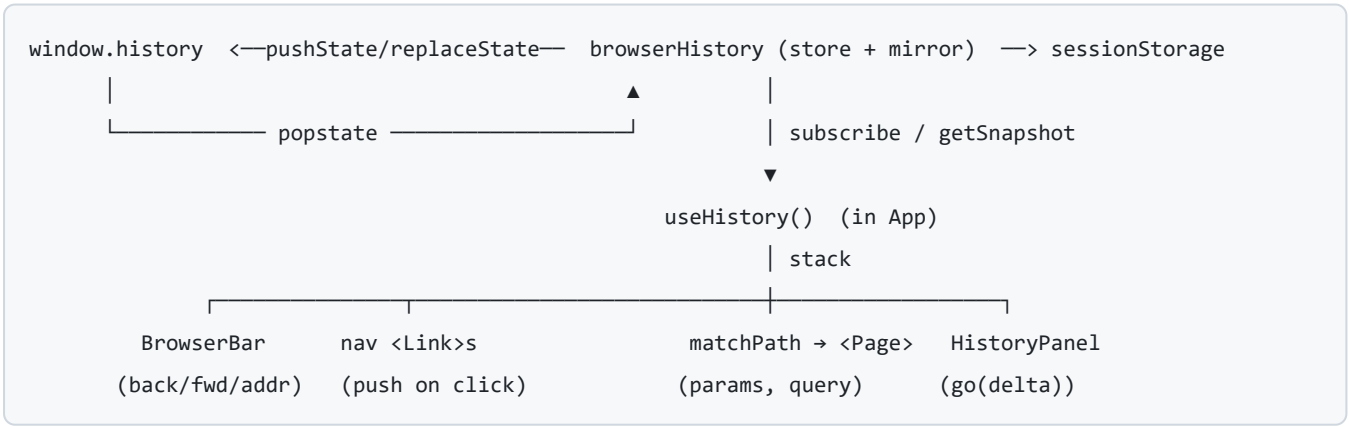
Action / input	Expected behavior / output
First load on `/`	Home page, history = `[/]`, Back and Forward disabled
Click "Products" link	URL `/products`, Products page, no reload, new entry, Back enabled
Click a product	URL `/products/2`, detail page, new entry
Click in-app Back (←)	URL `/products`, Forward enabled, current entry moves up in the panel
Press the browser's real Back button	Same as in-app Back (both go through `popstate`)
Visit a new page after going back	Forward entries are removed, Forward disabled
Click the link of the page you are on	Nothing happens, no duplicate entry
Ctrl/Cmd-click or middle-click a link	Browser opens a new tab; the app does not navigate
Click an entry in the history panel	Jump there with `history.go(delta)`; works both ways
Type `products/4` + Enter in the address bar	Normalized to `/products/4`, navigates
Type `https://example.com` + Enter	Inline error "Enter an in-app path", no navigation
Type in product search	URL becomes `/products?q=ph`, list filters, **no new history entries**
Search with no match	"No products match ..." message

Action / input	Expected behavior / output
Back to a search page	Search box and list come back from the URL
Unknown path `/does-not-exist`	404 page with a "Go home" link
`/products/999`	404 (product not found)
Reload (button or F5)	Same page, same history list, same current index

This table is the implementation checklist and the test plan.

3. Design the architecture

Module	Responsibility
`historyStack.js`	Pure functions over `{ entries, index }`: `visit`, `replace`, `moveTo`, `canGoBack`, `canGoForward`
`browserHistory.js`	Adapter + store: calls `pushState` / `replaceState`, listens to `popstate`, keeps the mirror, saves it to `sessionStorage`, notifies subscribers
`useHistory.js`	React hook: subscribes to the store with `useSyncExternalStore`
`matchPath.js`	Pure: finds the route for a pathname and extracts `:params`
`Link.jsx`	`<a href>` that intercepts plain left clicks and calls `push` / `replace`
`App.jsx`	Route table; derives the current URL; renders toolbar, nav, page, panel
`BrowserBar.jsx`	Back / Forward / Reload buttons + address bar with validation
`HistoryPanel.jsx`	Renders the mirrored stack; click → `go(delta)`
`pages.jsx`	Home, Products (search), ProductDetail, About, NotFound



Start simple: one store module and one hook. No context provider is needed because the store is a module singleton (there is only one `window.history`).

4. Design the state / data model

State or field	Purpose
`stack.entries` (store)	Ordered list of paths in this tab, mirror of the real history
`stack.index` (store)	Position of the current entry; also written into `history.state.idx`
`history.state = { idx }` (browser)	Tag on each real entry so `popstate` tells us where we landed
`sessionStorage['mini-browser-history']`	Copy of the stack, so it survives reload
`draft` (AddressBar)	What the user is typing before Enter
`error` (AddressBar)	Validation message for the address bar

```
// browserHistory.js (module state)
let stack = restore() ?? { entries: [currentPath()], index: 0 };
history.replaceState({ idx: stack.index }, '');
const listeners = new Set();

// App.jsx
const stack = useHistory();
const url = new URL(stack.entries[stack.index], location.origin); // derived
const match = matchPath(routes, url.pathname);                    // derived
const q = url.searchParams.get('q') ?? '';                        // derived (in Products)
```

Senior note: derive, don't store.

- The **current location**, **route params**, **search query**, **canGoBack / canGoForward**, and the **back / forward counts** are all derived from `{ entries, index }`. Storing any of them separately would let them drift out of sync.
- The search box has **no** `useState`: its value is read from the URL. That is why back / forward / reload restore it with zero extra code.
- **Ownership:** the browser owns the real history; `browserHistory` owns the mirror; `AddressBar` owns only its draft and error (local UI state). The draft is reset with `key={current}` instead of a sync effect.

5. Design the data structures and contracts

Example data

```
// the mirrored stack after: / → /products → /products/2 → Back
{ entries: ['/', '/products', '/products/2'], index: 1 }

// what the browser stores on the entry for /products
history.state // { idx: 1 }

// products (stable string ids because they come from the URL, which is always a string)
[{ id: '1', name: 'Laptop', price: 999 }, { id: '2', name: 'Phone', price: 699 }, ...]
```

Entries are plain strings. The same path can appear twice (`/ → /a → /`), so the **position** is the identity of an entry, and the index is a correct React key for the panel list.

Contracts

Function	Input	Output / effect
`visit(stack, path)`	stack, path	New stack with forward entries dropped and `path` appended; same object if `path` is current
`replace(stack, path)`	stack, path	New stack with the current entry swapped
`moveTo(stack, index, path)`	stack, index from `popstate`, actual URL	New stack with the cursor moved
`browserHistory.push(path)`	`'/products/2'`	`pushState({ idx }, '', path)`, scroll top, notify
`browserHistory.replace(path)`	`'/products?q=ph'`	`replaceState({ idx }, '', path)`, notify
`browserHistory.back() / forward() / go(n)`	– / delta	Calls the browser; the UI updates later on `popstate`
`browserHistory.subscribe(fn)`	listener	Returns `unsubscribe`
`browserHistory.getSnapshot()`	–	Current `{ entries, index }` (same object until it changes)
`matchPath(routes, pathname)`	`[{ path: '/products/:id' }]`, `'/products/2'`	`{ route, params: { id: '2' } }` or `null`
`<Link to replace? />`	path, optional replace	Renders `<a href>`; plain left click → push / replace

There is no server API; the product list is a local mock.

6. Trace the data flow

User clicks "Phone", then the browser Back button:

```
click <a href="/products/2">            (Link.handleClick)
  ↓ plain left click? yes → event.preventDefault()   (no page load)
browserHistory.push('/products/2')
  ↓ visit(stack, path) → { entries: ['/', '/products', '/products/2'], index: 2 }
  ↓ history.pushState({ idx: 2 }, '', '/products/2')  (URL bar changes, no request)
  ↓ scrollTo(0, 0); sessionStorage.setItem(...); listeners.forEach(fn)
useSyncExternalStore sees a new snapshot → App re-renders
  ↓ url = /products/2 → matchPath → ProductDetail({ id: '2' }) → "Phone"
  ↓ Back enabled, panel highlights entry 2

user presses the browser Back button
  ↓ browser restores the previous entry, fires popstate with event.state = { idx: 1 }
popstate handler: moveTo(stack, 1, '/products') → commit → re-render
  ↓ Products page, Forward enabled, entry 2 greyed in the panel
```

7. Important technical challenges

Challenge 1: the browser hides its history list and back/forward are asynchronous

Problem: `window.history` only gives `length` and `state`. You cannot read the URLs, and you cannot tell whether a `popstate` was Back or Forward or a jump of 3. Also `history.back()` returns immediately; the URL changes later. If you update React state right after calling `back()`, you read the old URL.

Solution: keep a mirror `{ entries, index }` and write the index into each entry's `state`. On `popstate`, `event.state.idx` tells you exactly where you are. Never update state in `back()`; let `popstate` be the single place that handles every move (in-app buttons, browser buttons, keyboard, `go(n)`).

```
history.replaceState({ idx: stack.index }, '');  // tag the landing entry

window.addEventListener('popstate', (event) => {
  const idx = event.state?.idx;
  if (Number.isInteger(idx) && idx < stack.entries.length) {
    commit(Stack.moveTo(stack, idx, currentPath()));
    return;
  }
  const next = Stack.visit(stack, currentPath()); // untagged entry, e.g. a #hash link
  history.replaceState({ idx: next.index }, '');
  commit(next);
});

back: () => history.back(), // async: UI updates in popstate
```

Challenge 2: new visit truncates forward, duplicates, and replace vs push

Problem: `/ → /a → /b`, Back twice, visit `/c`. A real browser now has `[/, /c]`: `/a` and `/b` are gone. If the mirror just appends, it shows entries the browser no longer has. Clicking the link of the current page should not add a copy. Typing "phone" in search with `pushState` would add 5 entries; the user then needs 5 Backs to leave the page.

Solution: `visit` slices to `index + 1` before appending and returns the same object for the same URL (so `push` can skip `pushState`). Search uses `replaceState`, which swaps the current entry and keeps the forward stack.

```
export function visit(stack, path) {
  if (stack.entries[stack.index] === path) return stack;
  const entries = [...stack.entries.slice(0, stack.index + 1), path];
  return { entries, index: entries.length - 1 };
}

export function replace(stack, path) {
  return { entries: stack.entries.with(stack.index, path), index: stack.index };
}
```

Challenge 3: intercepting link clicks without breaking the browser

Problem: If links are `<button onClick={navigate}>`, users lose "open in new tab", copy link, middle-click, and screen readers announce a button. If every `<a>` click is intercepted, Ctrl+click opens the page in the same tab.

Solution: render a real `<a href>` and only call `preventDefault()` for a plain left click with no modifier keys and no `target="_blank"`.

```
function handleClick(event) {
  onClick?.(event);
  const newTabIntent = event.button !== 0 || event.metaKey || event.ctrlKey || event.shiftKey ||
event.altKey;
  if (event.defaultPrevented || newTabIntent || rest.target === '_blank') return;
  event.preventDefault();
  if (replace) browserHistory.replace(to);
  else browserHistory.push(to);
}
```

Challenge 4: surviving reload and subscribing React to an outside store

Problem: On F5 the JS memory is gone but the browser still has the entries and their `state`. Without care, the mirror resets to one entry while Back still works, so the panel lies. Also, a `useState` copied from `window.history` in a `useEffect` can tear (render with old data) and needs manual cleanup.

Solution: save the mirror to `sessionStorage` (per tab, survives reload, same lifetime as the tab's history). On start, trust it only if the saved entry at `history.state.idx` equals the current URL. Subscribe with `useSyncExternalStore`, which is built for external stores.

```
function restore() {
  try {
    const saved = JSON.parse(sessionStorage.getItem(STORAGE_KEY));
    const idx = history.state?.idx;
    if (saved && saved.entries[idx] === currentPath()) return { entries: saved.entries, index: idx };
  } catch { /* storage blocked or corrupt: start fresh */ }
  return null;
}

export function useHistory() {
  return useSyncExternalStore(browserHistory.subscribe, browserHistory.getSnapshot);
}
```

8. Implement in the correct order

#	Step	Working result
1	Vite + Tailwind scaffold, static Home page	App renders
2	`historyStack.js` (`visit` , `replace` , `moveTo` , `canGo*`) + quick asserts	Stack logic proven without a browser
3	`browserHistory.push` + `pushState` , `subscribe` / `getSnapshot`	URL changes on push
4	`useHistory` hook, App reads the current path	Page text follows the URL

#	Step	Working result
5	`matchPath` + route table + 404	`/products/:id` and unknown paths work
6	`Link` with click interception	Navigation without reload, Ctrl+click still opens a tab
7	`popstate` listener + `{ idx }` tagging	Browser Back / Forward work
8	BrowserBar: Back / Forward (disabled states), Reload	In-app controls work
9	HistoryPanel with `go(delta)`	Visual stack, jump to any entry
10	AddressBar with normalize + error	Typed navigation, external URLs rejected
11	Products search with `replace` and `?q=`	Search in URL, no history flood
12	`sessionStorage` save + `restore()`	Reload keeps the history list

9. Cover all states and edge cases

State / edge case	Handling
Initial load at `/`	Stack `[/]`, Back and Forward disabled
Initial load at a deep URL (`/products/3`)	Stack starts at that path; works because the server serves `index.html` for every path
At the first entry	Back disabled (`index === 0`)
At the last entry	Forward disabled
Middle of the stack	Both enabled; panel greys forward entries
New visit from the middle	Forward entries dropped
Same URL clicked again	No `pushState`, no re-render
Modifier / middle click, `target="_blank"`	Browser default (new tab)
Unknown route	`NotFound` page
Valid route, unknown id (`/products/999`)	`NotFound` page
Encoded params (`/products/a%20b`)	`decodeURIComponent` → `a b`
Empty search	URL back to `/products`, full list
Search with no matches	Empty-state message
Address bar empty or other origin	Inline `role="alert"` error, no navigation
Address bar without leading slash	Normalized (`about` → `/about`)
Reload	Mirror restored from `sessionStorage`

State / edge case	Handling
`sessionStorage` blocked / corrupted JSON	`try/catch`, start fresh with one entry
`popstate` on an entry we never tagged (`#hash` link)	Treated as a new visit and tagged
Fast repeated Back clicks	Each fires its own `popstate`; the last one wins, state stays consistent because it comes from `event.state`
New page scroll	`scrollTo(0, 0)` on push; Back / Forward keep the native scroll restoration

10. Final structure

```
src/
├── main.jsx
├── index.css
├── App.jsx                    route table, derives URL, layout
├── history/
│   ├── historyStack.js        pure stack functions
│   ├── browserHistory.js      History API adapter + store + persistence
│   └── useHistory.js          useSyncExternalStore hook
├── router/
│   ├── matchPath.js           pure route matcher
│   └── Link.jsx               intercepting <a>
├── components/
│   ├── BrowserBar.jsx         back / forward / reload / address bar
│   └── HistoryPanel.jsx       visual stack
├── pages/
│   └── pages.jsx              Home, Products, ProductDetail, About, NotFound
└── data/
    └── products.js            mock data
```

11. Full working code

Setup: `npm create vite@latest` (React, JavaScript), then `npm i tailwindcss @tailwindcss/vite` and add `tailwindcss()` to the plugins in `vite.config.js`.

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/index.css

```
@import "tailwindcss";
```

src/history/historyStack.js

```
// Pure model of one tab's history: a list of URLs and a cursor. No DOM here, so it is easy to test.

export const createStack = (path) => ({ entries: [path], index: 0 });

export const canGoBack = (stack) => stack.index > 0;
export const canGoForward = (stack) => stack.index < stack.entries.length - 1;

// New visit: drop every forward entry, then append. Same URL as current = no new entry.
export function visit(stack, path) {
  if (stack.entries[stack.index] === path) return stack;
  const entries = [...stack.entries.slice(0, stack.index + 1), path];
  return { entries, index: entries.length - 1 };
}

// Swap the current entry in place (no new entry, forward stack kept).
export function replace(stack, path) {
  return { entries: stack.entries.with(stack.index, path), index: stack.index };
}

// Browser moved the cursor (back / forward / go). It owns the real stack, we only mirror it.
export function moveTo(stack, index, path) {
  return { entries: stack.entries.with(index, path), index };
}
```

src/history/browserHistory.js

```
import * as Stack from './historyStack';

// Adapter over window.history. The browser hides its entry list, so we mirror it
// and tag every entry with its position: history.state = { idx }.
const STORAGE_KEY = 'mini-browser-history';
const currentPath = () => location.pathname + location.search;

function restore() {
  try {
    const saved = JSON.parse(sessionStorage.getItem(STORAGE_KEY));
    const idx = history.state?.idx;
    if (saved && saved.entries[idx] === currentPath()) return { entries: saved.entries, index: idx };
  } catch {
    // storage blocked or corrupt: start fresh
  }
  return null;
}

let stack = restore() ?? Stack.createStack(currentPath());
history.replaceState({ idx: stack.index }, ''); // tag the entry we landed on
const listeners = new Set();

function commit(next) {
  stack = next;
  try {
    sessionStorage.setItem(STORAGE_KEY, JSON.stringify(stack)); // same lifetime as the tab's history
  } catch {
    // shortcut: mirror is lost on reload if storage is blocked; acceptable for a demo
  }
  listeners.forEach((listener) => listener());
}

// Fires on browser back/forward, history.go() and our own back()/forward() buttons.
window.addEventListener('popstate', (event) => {
  const idx = event.state?.idx;
  if (Number.isInteger(idx) && idx < stack.entries.length) {
    commit(Stack.moveTo(stack, idx, currentPath()));
    return;
  }
  // Entry we never tagged (e.g. a #hash link or a lost mirror): treat it as a new visit.
  const next = Stack.visit(stack, currentPath());
  history.replaceState({ idx: next.index }, '');
  commit(next);
});
```

```
export const browserHistory = {
  subscribe(listener) {
    listeners.add(listener);
    return () => listeners.delete(listener);
  },
  getSnapshot: () => stack,

  push(path) {
    const next = Stack.visit(stack, path);
    if (next === stack) return; // same URL: no duplicate entry
    history.pushState({ idx: next.index }, '', path);
    window.scrollTo(0, 0); // new page starts at the top; back/forward use native scroll restoration
    commit(next);
  },
  replace(path) {
    history.replaceState({ idx: stack.index }, '', path);
    commit(Stack.replace(stack, path));
  },
  // Async: the UI updates when popstate fires, not here.
  back: () => history.back(),
  forward: () => history.forward(),
  go: (delta) => delta !== 0 && history.go(delta),
};
```

src/history/useHistory.js

```
import { useSyncExternalStore } from 'react';
import { browserHistory } from './browserHistory';

// Re-renders on every push / replace / popstate. Returns { entries, index }.
export function useHistory() {
  return useSyncExternalStore(browserHistory.subscribe, browserHistory.getSnapshot);
}
```

src/router/matchPath.js

```
// Finds the first route whose pattern ("/products/:id") matches the pathname.
export function matchPath(routes, pathname) {
  const parts = pathname.split('/').filter(Boolean);
  for (const route of routes) {
    const pattern = route.path.split('/').filter(Boolean);
    if (pattern.length !== parts.length) continue;
    const params = {};
    const ok = pattern.every((segment, i) => {
      if (segment.startsWith(':')) {
        params[segment.slice(1)] = decodeURIComponent(parts[i]);
        return true;
      }
      return segment === parts[i];
    });
    if (ok) return { route, params };
  }
  return null;
}
```

src/router/Link.jsx

```
import { browserHistory } from '../history/browserHistory';

// A real <a href> (so open-in-new-tab, copy link, SEO work), with the click taken over for in-app
navigation.
export default function Link({ to, replace = false, onClick, ...rest }) {
  function handleClick(event) {
    onClick?.(event);
    const newTabIntent = event.button !== 0 || event.metaKey || event.ctrlKey || event.shiftKey ||
event.altKey;
    if (event.defaultPrevented || newTabIntent || rest.target === '_blank') return; // let the browser
handle it
    event.preventDefault();
    if (replace) browserHistory.replace(to);
    else browserHistory.push(to);
  }

  return <a {...rest} href={to} onClick={handleClick} />;
}
```

src/data/products.js

```
export const products = [
  { id: '1', name: 'Laptop', price: 999 },
  { id: '2', name: 'Phone', price: 699 },
  { id: '3', name: 'Headphones', price: 199 },
  { id: '4', name: 'Keyboard', price: 89 },
  { id: '5', name: 'Monitor', price: 299 },
];
```

src/components/BrowserBar.jsx

```
import { useState } from 'react';
import { browserHistory } from '../history/browserHistory';
import { canGoBack, canGoForward } from '../history/historyStack';

// Turns what the user typed into an in-app path, or null if it points elsewhere.
function toPath(input) {
  const text = input.trim();
  if (!text) return null;
  try {
    const url = new URL(text.startsWith('/') || text.includes('://') ? text : `/${text}`,
location.origin);
    return url.origin === location.origin ? url.pathname + url.search : null;
  } catch {
    return null;
  }
}

const button =
  'h-9 w-9 rounded-md border border-slate-300 bg-white text-lg disabled:cursor-not-allowed
disabled:opacity-40 hover:enabled:bg-slate-100';

export default function BrowserBar({ stack }) {
  const current = stack.entries[stack.index];
  return (
    <div className="flex items-center gap-2 border-b border-slate-300 bg-slate-100 p-3">
      <button className={button} aria-label="Back" disabled={!canGoBack(stack)} onClick=
{browserHistory.back}>
        ←
      </button>
      <button className={button} aria-label="Forward" disabled={!canGoForward(stack)} onClick=
{browserHistory.forward}>
        →
      </button>
      <button className={button} aria-label="Reload" onClick={() => location.reload()}>
        ⟳
      </button>
      {/* key: reset the draft whenever the real URL changes */}
      <AddressBar key={current} current={current} />
    </div>
  );
}

function AddressBar({ current }) {
  const [draft, setDraft] = useState(current);
  const [error, setError] = useState('');
```

```
  function handleSubmit(event) {
    event.preventDefault();
    const path = toPath(draft);
    if (!path) return setError('Enter an in-app path, e.g. /products/2');
    browserHistory.push(path);
  }

  return (
    <form onSubmit={handleSubmit} className="relative flex-1">
      <input
        aria-label="Address"
        aria-invalid={Boolean(error)}
        value={draft}
        onChange={(e) => {
          setDraft(e.target.value);
          setError('');
        }}
        onFocus={(e) => e.target.select()}
        className="h-9 w-full rounded-full border border-slate-300 bg-white px-4 font-mono text-sm
outline-none focus:border-blue-500"
      />
      {error && (
        <p role="alert" className="absolute top-10 left-4 text-xs text-red-600">
          {error}
        </p>
      )}
    </form>
  );
}
```

src/components/HistoryPanel.jsx

```
import { browserHistory } from '../history/browserHistory';

// Shows the mirrored stack. Clicking an entry jumps there with history.go(delta).
export default function HistoryPanel({ stack }) {
  const { entries, index } = stack;
  return (
    <aside className="rounded-lg border border-slate-300 bg-white p-4">
      <h2 className="font-semibold">History ({entries.length})</h2>
      <p className="mb-3 text-xs text-slate-500">
        {index} back · {entries.length - 1 - index} forward
      </p>
      <ol className="space-y-1">
        {/* entries are positional and can repeat, so the index is their identity */}
        {entries.map((path, i) => (
          <li key={i}>
            <button
              onClick={() => browserHistory.go(i - index)}
              aria-current={i === index ? 'page' : undefined}
              className={`w-full truncate rounded px-2 py-1 text-left font-mono text-sm ${
                i === index ? 'bg-blue-600 text-white' : i > index ? 'text-slate-400 hover:bg-slate-
100' : 'hover:bg-slate-100'
              }`}
            >
              {i}. {path}
            </button>
          </li>
        ))}
      </ol>
    </aside>
  );
}
```

src/pages/pages.jsx

```
import Link from '../router/Link';
import { browserHistory } from '../history/browserHistory';
import { products } from '../data/products';

const link = 'text-blue-600 underline';

export function Home() {
  return (
    <>
      <h1 className="text-2xl font-bold">Home</h1>
      <p>Click around, then use the ← → buttons, the real browser buttons, or the history panel.</p>
      <Link to="/products/2" className={link}>
        Jump to Phone
      </Link>
    </>
  );
}

export function Products({ query }) {
  // The search box is driven by the URL (?q=), so back/forward/reload restore it for free.
  const q = query.get('q') ?? '';
  const visible = products.filter((p) => p.name.toLowerCase().includes(q.toLowerCase()));

  function handleSearch(event) {
    const value = event.target.value;
    // replace, not push: one entry per search, not one per keystroke
    browserHistory.replace(value ? `/products?q=${encodeURIComponent(value)}` : '/products');
  }

  return (
    <>
      <h1 className="text-2xl font-bold">Products</h1>
      <input
        aria-label="Search products"
        placeholder="Search…"
        value={q}
        onChange={handleSearch}
        className="rounded border border-slate-300 px-3 py-1"
      />
      {visible.length === 0 ? (
        <p className="text-slate-500">No products match “{q}”.</p>
      ) : (
        <ul className="list-disc pl-6">
          {visible.map((p) => (
            <li key={p.id}>
```

```
              <Link to={`/products/${p.id}`} className={link}>
                {p.name}
              </Link>
            </li>
          ))}
        </ul>
      )}
    </>
  );
}

export function ProductDetail({ params }) {
  const product = products.find((p) => p.id === params.id);
  if (!product) return <NotFound />;
  return (
    <>
      <h1 className="text-2xl font-bold">{product.name}</h1>
      <p>${product.price}</p>
      <Link to="/products" className={link}>
        All products
      </Link>
    </>
  );
}

export function About() {
  return (
    <>
      <h1 className="text-2xl font-bold">About</h1>
      <p>A tiny router built on pushState, replaceState and popstate.</p>
    </>
  );
}

export function NotFound() {
  return (
    <>
      <h1 className="text-2xl font-bold">404: Page not found</h1>
      <Link to="/" className={link}>
        Go home
      </Link>
    </>
  );
}
```

src/App.jsx

```
import { useHistory } from './history/useHistory';
import { matchPath } from './router/matchPath';
import Link from './router/Link';
import BrowserBar from './components/BrowserBar';
import HistoryPanel from './components/HistoryPanel';
import { Home, Products, ProductDetail, About, NotFound } from './pages/pages';

const routes = [
  { path: '/', page: Home, label: 'Home' },
  { path: '/products', page: Products, label: 'Products' },
  { path: '/products/:id', page: ProductDetail },
  { path: '/about', page: About, label: 'About' },
];

export default function App() {
  const stack = useHistory();
  const url = new URL(stack.entries[stack.index], location.origin); // current location is derived, not
stored
  const match = matchPath(routes, url.pathname);
  const Page = match?.route.page ?? NotFound;

  return (
    <div className="min-h-screen bg-slate-50 text-slate-800">
      <BrowserBar stack={stack} />
      <div className="mx-auto grid max-w-5xl gap-6 p-6 md:grid-cols-[1fr_280px]">
        <main className="space-y-4">
          <nav className="flex gap-4 border-b border-slate-200 pb-3">
            {routes
              .filter((r) => r.label)
              .map((r) => (
                <Link
                  key={r.path}
                  to={r.path}
                  aria-current={url.pathname === r.path ? 'page' : undefined}
                  className="aria-[current=page]:font-bold aria-[current=page]:text-blue-600"
                >
                  {r.label}
                </Link>
              ))}
            <Link to="/does-not-exist" className="text-slate-400">
              Broken link
            </Link>
          </nav>
          <Page params={match?.params ?? {}} query={url.searchParams} />
        </main>
```

```
        <HistoryPanel stack={stack} />
      </div>
    </div>
  );
}
```

check.mjs: self-check of the pure logic (project root, run with `node check.mjs`)

```
import assert from 'node:assert/strict';
import * as S from './src/history/historyStack.js';
import { matchPath } from './src/router/matchPath.js';

let s = S.createStack('/');
s = S.visit(s, '/a');
s = S.visit(s, '/b');
assert.deepEqual(s, { entries: ['/', '/a', '/b'], index: 2 });
assert.equal(S.visit(s, '/b'), s, 'same URL adds no entry');
s = S.moveTo(s, 0, '/'); // back twice
assert.ok(!S.canGoBack(s) && S.canGoForward(s));
s = S.visit(s, '/c'); // new visit drops forward entries
assert.deepEqual(s, { entries: ['/', '/c'], index: 1 });
assert.ok(!S.canGoForward(s));
s = S.moveTo(s, 0, '/');
s = S.replace(s, '/?q=x'); // replace keeps forward stack
assert.deepEqual(s, { entries: ['/?q=x', '/c'], index: 0 });

const routes = [{ path: '/' }, { path: '/products' }, { path: '/products/:id' }];
assert.equal(matchPath(routes, '/').route.path, '/');
assert.deepEqual(matchPath(routes, '/products/a%20b').params, { id: 'a b' });
assert.equal(matchPath(routes, '/products/1/x'), null);
assert.equal(matchPath(routes, '/nope'), null);
console.log('self-check ok');
```

12. Wrap-up

Edge cases handled

- Back / Forward from both the app and the real browser, through one `popstate` path.
- Forward truncation on a new visit, no duplicate entries for the same URL.
- Modifier clicks and `target="_blank"` left to the browser.
- 404 for unknown routes and unknown ids; decoded params.
- Search in the URL with `replaceState` (no history flood), restored by Back and reload.
- Address bar normalization and same-origin validation.
- Reload restore with a guard against a stale or corrupt mirror.

Trade-offs to mention

- **Mirror vs truth:** we keep our own copy of the stack because the browser hides it. It can drift if something else calls `pushState` (a third-party script). The guard in `popstate` heals the common cases.
- **Module singleton store** instead of Context: simpler and correct because there is only one `window.history`; harder to test with several instances (you would pass a `history` object in for tests).
- `sessionStorage` instead of `localStorage`: history is per tab, so per-tab storage is the right lifetime.
- **Linear route matching:** fine for tens of routes; a trie or precompiled regex helps at hundreds.
- **Search on every keystroke** with `replaceState`: simple; browsers throttle very frequent `pushState` / `replaceState` calls (Safari throws after too many), so debounce for heavy typing.
- **Server config:** deep links need the server to serve `index.html` for unknown paths (SPA fallback). Hash routing avoids that but gives ugly URLs.

What to add with more time

- Unit tests for `historyStack` and `matchPath` (Vitest), and Playwright tests for real Back / Forward / reload.
- Navigation guards ("You have unsaved changes"): a `block()` that runs before push and on `beforeunload`.
- Focus management: move focus to the page `<h1>` after navigation and announce the new page title for screen readers; update `document.title` per route.
- Custom scroll restoration per entry (save `scrollY` in `history.state`) for pages that load data asynchronously.
- Nested routes, layouts, lazy-loaded pages (`React.lazy`) with a loading state.
- The newer **Navigation API** (`navigation.navigate`, `navigate` event, `navigation.entries()`) which exposes the entry list directly; fall back to this code where it is not supported.

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just use React Router?

In a real product I would. Here the goal is to show how it works: `pushState` to change the URL, `popstate` to react to Back / Forward, and click interception on links. React Router's `BrowserRouter` is these same pieces plus nested routes, loaders and guards.

Q2. What is the difference between `pushState`, `replaceState` and setting `location.href`?

`pushState` adds a new entry and changes the URL without a request. `replaceState` changes the current entry (no new entry). `location.href = ...` does a full page load and throws away the JS state. Neither `pushState` nor `replaceState` fires `popstate`.

Q3. Why does `popstate` not fire when I call `pushState`?

By spec, `popstate` fires only when the browser moves between entries (Back, Forward, `go`, or fragment navigation). Your own `pushState` is something you already know about, so you update the UI yourself right

after it. That is why `push` calls `commit` directly.

State design

Q4. Why store `idx` in `history.state` instead of comparing URLs?

The same URL can appear many times in the stack (`/ → /a → /`). Comparing URLs cannot tell which `/` you landed on, or whether you went back or forward. The index is unique per position, so `event.state.idx` gives an exact answer.

Q5. Why is the search query not in `useState`?

The URL is already the source of truth. A second copy in state would need syncing on Back / Forward / reload and could drift. Reading `searchParams.get('q')` makes all three work for free.

Q6. Why `useSyncExternalStore` instead of `useState` + `useEffect`?

The history lives outside React. `useSyncExternalStore` subscribes, reads a snapshot during render, and prevents tearing in concurrent rendering. With `useEffect` you get one render with stale data and must handle cleanup yourself. `getSnapshot` must return the same object until something changes, which our immutable updates guarantee.

Q7. Why is the index a valid React key in the history panel?

Entries are positional and can repeat, so the position is their identity. Entries are only appended or cut from the end, never reordered or inserted in the middle, so an index key never points to the wrong item.

Challenge 1: async back / forward

Q8. What happens if I read `location.pathname` right after `history.back()`?

You get the old path. `back()` is asynchronous; the URL changes and `popstate` fires a moment later. That is why the UI updates only in the `popstate` handler.

Q9. How would you know whether the user went Back or Forward?

Compare `event.state.idx` with the previous `stack.index`: smaller means back, bigger means forward, and the difference is the number of steps. This is useful for slide animations in the right direction.

Challenge 2: truncation and replace

Q10. Why not `pushState` on every keystroke in search?

Typing "phone" would add 5 entries, and the user would need 5 Back presses to leave the page. `replaceState` keeps one entry that always holds the latest query. A common variant is: push once when the search starts, replace while typing.

Q11. What if the user clicks the link to the page they are already on?

`visit` returns the same stack object, `push` sees `next === stack` and skips `pushState`. No duplicate entry, no re-render. Some apps instead re-run the page (refresh data); that would be an explicit option.

Challenge 3: link interception

Q12. Why a real `<a href>` and not a `<button>` with `onClick`?

A real link supports open in new tab, copy link address, middle-click, the status-bar preview, SEO crawling and the right screen-reader role. We keep all of that and only take over a plain left click.

Q13. What about external links like `https://google.com` in `<Link>`?

Our `Link` assumes in-app paths. For safety it should check `new URL(to, location.origin).origin === location.origin` and fall back to a normal navigation otherwise, because `pushState` throws a `SecurityError` for another origin.

Challenge 4: reload and persistence

Q14. Why `sessionStorage` and not `localStorage`?

Browser history is per tab. `sessionStorage` is per tab and survives reload, exactly the same lifetime. With `localStorage`, two tabs would overwrite each other's history.

Q15. Can you store the whole page data in `history.state`?

You can store serializable data (it is structured-cloned), but browsers cap its size (Firefox throws above its limit). Store small keys (ids, scroll position) and keep big data in a cache keyed by them.

Implementation steps

Q16. Why is `browserHistory` a module singleton and not a Context?

There is exactly one `window.history` per page, so one store is the truth. A Context would add a provider without adding anything. For tests or SSR, I would turn it into `createBrowserHistory(window)` and pass a memory version in tests.

Q17. How does the 404 work on a refresh of `/products/2` in production?

The server must return `index.html` for any unknown path (SPA fallback: Nginx `try_files $uri /index.html`, Netlify `_redirects`, Vite does it in dev). Then the app reads `location.pathname` on start and `matchPath` picks the page. Without the fallback the server answers 404 before React runs.

Q18. How would you add a route guard, e.g. "unsaved changes"?

Add `block(fn)` to the store. `push` asks the blocker first. For Back / Forward you cannot cancel `popstate`, so you restore with `history.go(-delta)` when the blocker says no. For tab close and reload, use `beforeunload`.

Performance

Q19. What if there are 1,000 routes?

Linear matching is still under a millisecond, but you can precompile each pattern to a regex once, or build a trie by segment so a lookup is O(segments). Lazy-load page components so the bundle does not grow with the route count.

Q20. What if the user navigates 10,000 times in one tab?

The mirror grows and so does the `sessionStorage` write on each commit. Cap the mirror to the last N entries (and shift the `idx` values), or virtualize the history panel list. Writes stay cheap for a few thousand short strings.

Q21. What if a page is slow to load data (slow API)?

Navigation should still be instant: change the URL, show a loading state in the page, and cancel the old request with `AbortController` when the user navigates again so a slow response cannot overwrite the newer page.

Trade-offs

Q22. Why not hash routing (`/#/products`)?

Hash routing needs no server config and works on static hosts, but URLs look worse, the hash cannot be used for in-page anchors, and SEO is weaker. The History API gives clean URLs at the cost of the SPA fallback on the server.

Q23. What does the new Navigation API change?

`window.navigation` exposes `entries()` , `currentEntry.index` , and one `navigate` event for every navigation (including link clicks), so you need neither the mirror nor click interception. Support is not universal yet, so this History API version is the fallback.

Q24. What is the risk of mirroring the stack yourself?

If another script calls `pushState` without our `{ idx }` tag, or storage fails, the mirror can drift from the real stack. The `popstate` guard treats untagged entries as new visits, which heals the common case; the Navigation API removes the problem entirely.